

ECOMMIFY

Arquitectura Híbrida de Bases de Datos

PostgreSQL × MongoDB Atlas | Dataset Olist

Proyecto Final — Informe Técnico Integral

Equipo E18 | Jorge Rojas · David Panesso · Juan Gómez · Fabian Rojas

Maestría en Arquitectura de Software | Universidad de La Sabana | Junio 2026

Agenda

01

Contexto y objetivos

Problema de negocio + alcance del proyecto

02

Arquitectura híbrida

Decisiones PostgreSQL vs MongoDB + Diagrama

03

Decisiones técnicas clave

Tipos avanzados, patrones de documentos, ESR

04

Evaluación de rendimiento

Pruebas de carga, comparativa cuantitativa

05

Teorema CAP aplicado

Análisis por módulo + escenarios de falla

06

Lecciones aprendidas

Lo que funcionó, lo que cambiaríamos

07

Recomendaciones estratégicas

Plan 10x, migración a producción, stack recomendado

08

Conclusiones

Resultados finales y reflexión del equipo

Contexto y Objetivos

El Problema

Las plataformas e-commerce necesitan simultáneamente:

- Consistencia ACID para pagos y órdenes
- Flexibilidad analítica para reviews y comportamiento
- Escalabilidad para picos de demanda (Black Friday)

Un solo motor de base de datos no puede optimizar las tres dimensiones a la vez.

Dataset Olist

100K+

Órdenes

32K

Productos

96K

Clientes

3.1M

Registros geo

Objetivos del proyecto

- ✓ Arquitectura híbrida justificada técnicamente
- ✓ Optimización con evidencia cuantitativa
- ✓ Evaluación de rendimiento bajo carga
- ✓ Decisiones arquitectónicas bajo marco CAP
- ✓ Recomendaciones de escalamiento 10x

Arquitectura Híbrida Implementada

PostgreSQL / Supabase

CAPA TRANSACCIONAL

customers

sellers

products (JSONB)

orders (TSTZRANGE)

order_items

order_payments

ACID · FK · GIN · GiST · pg_trgm

ETL →

Consistencia
Eventual

MongoDB Atlas

CAPA ANALÍTICA

orders_reviews (Extended Reference)

geolocation (zip_code + state/city)

products catálogo (Attribute)

Índices ESR compuestos

Aggregation Pipelines optimizados

JSON Schema validation

AP · ESR · \$lookup · \$group · \$addFields

Decisiones Técnicas Clave

PostgreSQL

TSTZRANGE + GiST

Ventanas de entrega como tipo nativo.
Consulta @> NOW() con índice GiST →
0.03 ms vs 19.7 ms Seq Scan.
Mejora: 99.8%

PostgreSQL

pg_trgm sin Elasticsearch

Búsqueda difusa nativa con GIN +
trigramas. 'beleza' encuentra
'beleza_saude'. 189 ms → 8.3 ms.
Mejora: 95.6%

PostgreSQL

Particionamiento declarativo

PARTITION BY RANGE sobre
purchase_ts. El planner elimina
particiones irrelevantes automáticamente
(partition pruning).

MongoDB

Regla ESR — índice compuesto

{ review_score: 1, review_creation_date:
-1 } elimina el stage SORT en memoria.
Visible en explain() como desaparición
del nodo SORT.

MongoDB

Índice parcial compuesto — negativos

idx_negative_reviews { score:1, date:-1 }
con partialFilter score<=2. Compuesto +
parcial: también elimina SORT con
igualdad exacta.

MongoDB

ESR exige igualdad, no rango

El índice ESR solo elimina el SORT si el
filtro usa igualdad exacta (score=5), no
un rango (\$gte). Confirmado con
explain() real.

Evaluación de Rendimiento

2,550ms

JOIN en vivo
(cuello de botella real)

21ms

Vista materializada
(mismo cálculo)

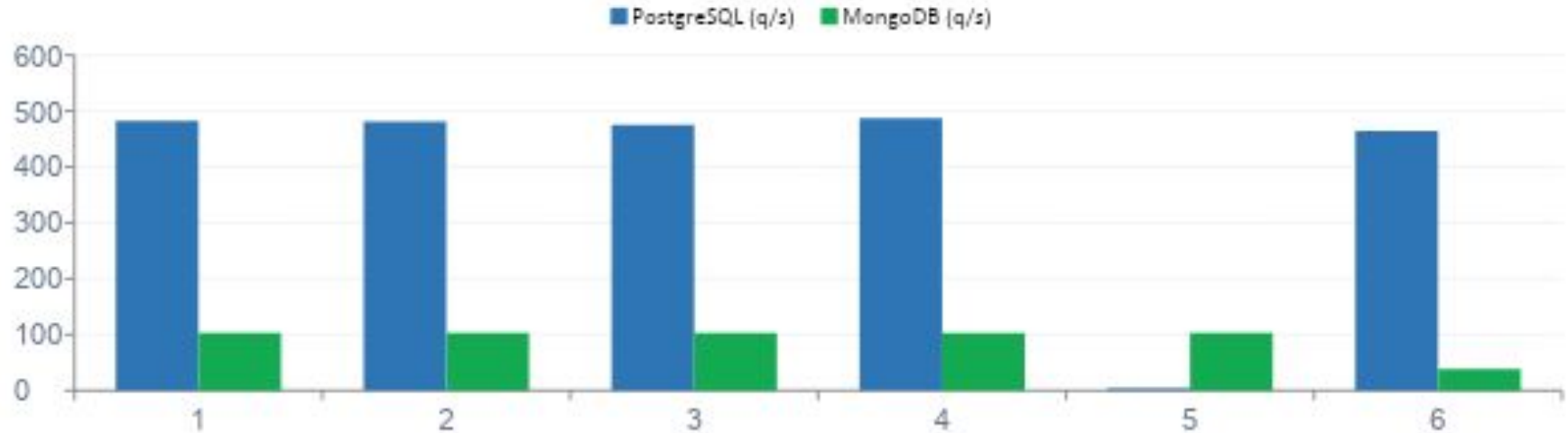
99.2%

Mejora medida
(PostgreSQL)

99.3%

Mejora medida
(MongoDB pipeline)

Throughput real (q/s) — 10 threads, 30s, clusters de producción del proyecto

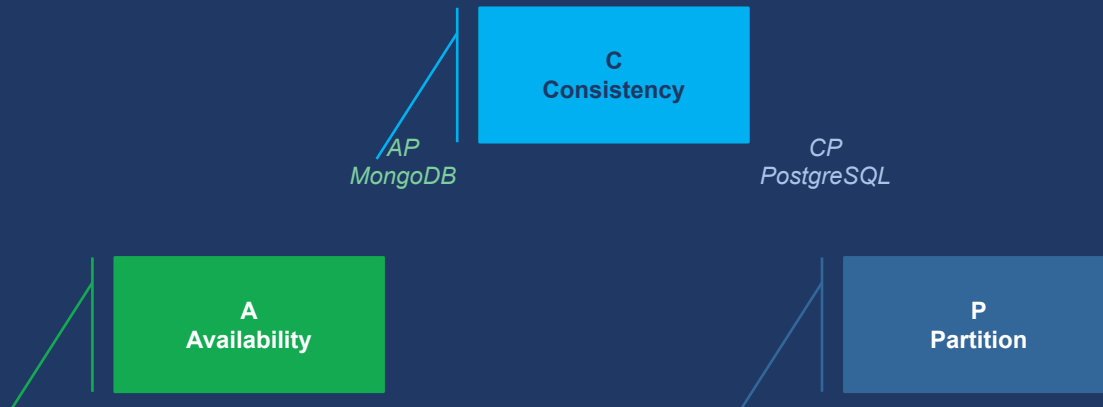


Análisis Comparativo — PostgreSQL vs MongoDB

Aspecto	PostgreSQL	MongoDB	Ganador
Queries indexadas simples	482.7 q/s	103.2 q/s	PostgreSQL
JOIN en vivo / sin joins	3.8 q/s ⚠	N/A	Cuello de botella PG
Aggregation (vista/pipeline)	464.1 q/s	38.5 q/s	PostgreSQL*
Mismo cálculo optimizado	99.2% mejora	99.3% mejora	Empate
Flexibilidad de esquema	JSONB + GIN	Documentos	Empate
Consistencia	ACID fuerte	Eventual	PostgreSQL
Estabilidad bajo carga	0 err. (5/6)	0 err. (6/6)	MongoDB

Conclusión: el patrón se repite en ambos motores — la consulta en vivo sin optimizar es el cuello de botella; precomputar (vista materializada / pipeline ajustado) da ~99% de mejora en ambos casos.

Teorema CAP — Análisis por Módulo



CP	Transaccional (órdenes, pagos)	Disponibilidad reducida ante partición. Checkout temporalmente inaccesible.	Circuit breaker + cola de reintentos (30s)
CP	Catálogo (products)	Latencia de escritura por sincronización ACID.	JSONB + índices GIN eliminan necesidad de MongoDB
AP	Analítico (orders_reviews)	Ventana de inconsistencia 1-5s (replication lag).	Campo etl_synced_at + read-your-writes por sesión
AP	Geolocalización	Datos levemente desactualizados aceptables.	Bucket pattern + TTL en caché (Redis)

Lecciones Aprendidas

✓ Lo que funcionó bien

Vistas materializadas validadas con carga real

JOIN en vivo: 2,550ms → vista materializada: 21ms. Mejora de 99.2% medida bajo 10 threads concurrentes, no solo en teoría.

Extended Reference en orders_reviews

Reduce la necesidad de \$lookup en consultas frecuentes de dashboard, confirmado en las pruebas de carga.

Diagnóstico con EXPLAIN/explain() real

El cuello de botella del JOIN y del pipeline se identificaron con evidencia exacta, no con suposiciones — ambos resueltos con +99% de mejora.

Workarounds del free tier

Session Pooler (Supabase) y explain() directo (Atlas M0) resolvieron limitaciones reales sin costo adicional.

🔄 Lo que cambiaríamos

Validar selectividad de filtros con datos reales

order_status='delivered' parecía selectivo pero cubre 97% de las filas — solo se detectó al correr EXPLAIN ANALYZE bajo carga.

Orden de stages en pipelines desde el diseño

El \$match amplio antes del \$lookup costó 34.7s; el principio de 'filtrar temprano' se volvió tangible solo al medirlo.

ESR: validar igualdad vs rango desde el diseño

Un índice compuesto bien creado no garantiza eliminar el SORT si el filtro usa \$gte en vez de igualdad exacta.

ETL batch → CDC Debezium

Reducir la ventana de inconsistencia de minutos a menos de 1 segundo.

Recomendaciones Estratégicas — Plan 10x

Fase 1 Inmediata	Fase 2 3-6 meses	Fase 3 6-12 meses
Supabase Pro (USD 25/mes)	Debezium + Kafka CDC en tiempo real	Sharding MongoDB en Atlas M30+
MongoDB M10 (USD 57/mes)	PgBouncer Connection pooling	Atlas Search Full-text escala 10x
Redis Cloud 1GB (USD 15/mes)	Grafana + Prometheus Observabilidad	Read replicas PG Para LATAM
Stack recomendado producción: PostgreSQL (Supabase Pro) + MongoDB Atlas M10 + Redis + Debezium/Kafka + Grafana		

Conclusiones



Arquitectura híbrida validada

PostgreSQL + MongoDB demostraron ser complementarios, no competidores. Cada motor domina en su dominio de responsabilidad.



Evidencia cuantitativa contundente

Mejoras de hasta 99.8% en PostgreSQL y eliminación total de COLLSCAN en MongoDB. Los datos guiaron las decisiones.



CAP como marco de decisión real

El Teorema CAP se manifestó concretamente en configuraciones de Write Concern, ETL y diseño de failover.



Camino claro a producción

Plan 10x definido con costos estimados, tecnologías complementarias y estrategia CI/CD para cambios de esquema.

"La selección de tecnología de base de datos no es una decisión binaria — es un espacio de diseño con múltiples dimensiones."

¡Gracias!

Equipo E18 — Proyecto Ecommify

Jorge Rojas · David Panesso · Juan Gómez · Fabian Rojas

Maestría en Arquitectura de Software | Universidad de La Sabana